SVI & MSX

— SPECTRAVIDEO

LIBRARY

NEWSLETTER

REGISTERED BY AUSTRALIA POST PUBLICATION No. TBH 0917 CATEGORY "B"

ISSUE NO.	ANNUAL SUBSCRIPTION		DATE
2 - 9	AUSTRALIA	$20.00	*JUNE - 85*
	OVERSEAS	$25.00	
	OVERSEAS AIRMAIL ...	$30.00	
	YEAR BOOK 83/84	$15.00	

CONTENTS

NEWSLETTER CORRESPONDENCE

S.A.U.G.,
P.O. BOX 191,
LAUNCESTON SOUTH,
TASMANIA, 7249.

(003) 312648

LIBRARY CORRESPONDENCE

S.A.U.G. LIBRARY,
1 CONRAD AVENUE,
GEORGE TOWN,
TASMANIA, 7253.

(003) 822919

Exploring Basic Pt-12

by L.A. Dunning.

This installment is the fourth part of a discussion about machine code routines (MCRs) and how to use them.

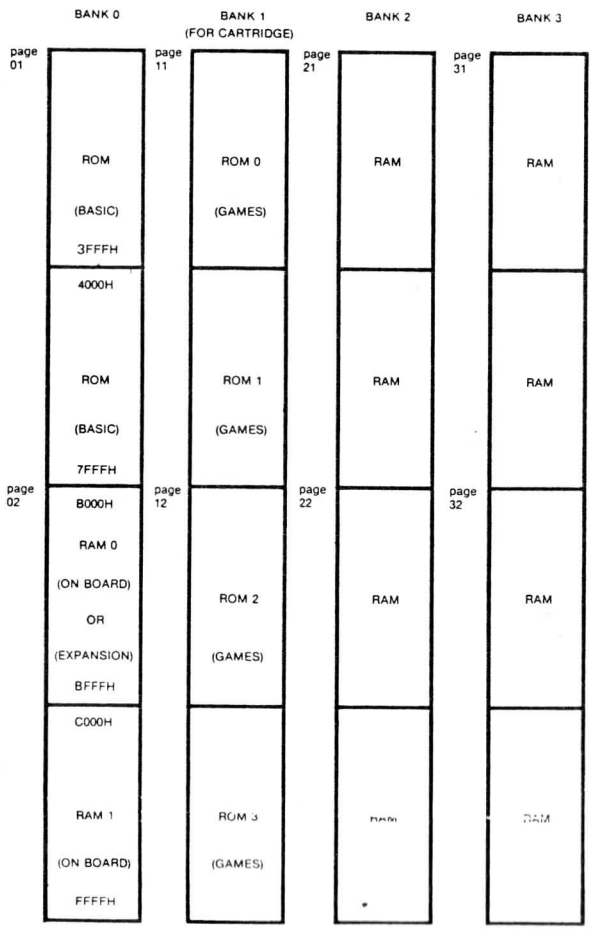
ROM ROUTINES

As you are probably already aware, the Spectravideo ROM is nothing more than a group of MCRs and tables as designed by MICROSOFT. On most computers you can't normally get rid of it, and it just sits in there eating up memory. While using basic proper, the ROM is neccesary to enable a program to be run. When using MCRs however there are two things you can do with the ROM: get rid of it or use it! The bulk of this part will deal with the first option and some spinoffs due to it.

MEMORY BANKS

You are probably aware of advertising claims that the SV has 80K of RAM and 32K of ROM, but may not be sure what it means. Examine Diagram 1. A normal unexanded SV318 comes with a ROM in page 01 and RAM in page 02 from C000H to FFFFH. An unexpanded SV328 comes with the above including the missing RAM from 8000H to BFFFH; it also has an extra 32K ram on page 21. In both cases there is an extra 16K video memory but this is "off-line" and can only be accessed via a port, as was demonstrated in earlier parts.

The RAM modules that plug into the expander add pages 22, 31 and 32. When an SV is turned on, pages 01 and 02 are normally active and the others inactive (henceforth called bank on/off). If the ROM discovers a CP/M disk is in a drive, it reads a bootstrap program into high memory and from there turns bank 21 on and then loads the rest of the system. Thus, an SV328 can run CP/M by



just adding the controller and drives - an SV318 requires the memory expansion to do so.

Assuming you own a 328 or expanded 318, you should be able to switch in and out various banks at will. The main advantage of this is that you can have a bigger program or routine to run, or an extra area to dump values or screen display. In basic (with the memory expansion) you can switch to an alternate bank by using the SWITCH statement. This was covered in an earlier installment. What SWITCH does is to turn page 22 on and repeat part of the initialisation process on that bank. It is possible to duplicate this procedure in an MCR.

Bank 0 is used by basic, bank 1 is used by the cartridge and bank 2 is used by CP/M. Bank 3 therefore is at the users disposal. There is a program available for the SV328 that converts it (mostly) to an MSX machine and this probably uses page 21 to store the new basic. This is a good example of the bank switching technique at work.

PORTS

We will now turn our attention to two devices hooked to the computer by several ports - the AY-3-8910 sound generating block (PSG) and the 8255 Programmable Peripheral Interface (PPI). Together these blocks access devices mostly used in games: sound chips, joysticks, et cetera.

The PSG is accessed via three ports; a "latch" port (88H), a read port (90H) and a write port (8CH). The PSG has 17 registers which are divided up as follows:

Register	Function
0-13	Sound Registers
14	Joysticks (PORT A)
15	Bank select (PORT B)
16	Unknown

The sound registers are the same registers that can be accessed using the SOUND statement in BASIC. Port A is used to input direction from the joysticks and port B selects which banks are used.

BIT	PORT A		PORT B	
0	Up	\	BANK 1	(Cartridge)
1	Down	\ STICK	Page 21	
2	Left	/ ONE	Page 22	
3	Right	/	Page 31	
4	Up	\	Page 32	
5	Down	\ STICK	Light	(Caps lock)
6	Left	/ TWO	ROM 2	(Page 12)
7	Right	/	ROM 3	(Page 12)

In Port A bits 0 to 3 indicate directions for joystick 1, bits 4 to 7 indicate directions for joystick 2. If a bit is set to "1" then there is no contact, whereas "0" indicates there is. If STICK 1 was moved in direction 2 and STICK was left untouched, port A would be 11110110B or F6H.

In port B, bit 0 enables the ROM Cartridge (bank 1). Bits 1 to 4 enable banks 2 and 3. Bits 6 and 7 enable the upper half of bank 1. In all cases a bit set to "0" is enabled and "1" is disabled. Bit 5 is used to set the CAPS LOCK light on however in this case "1" turns it on and "0" turns it off. A hardware disable ensures that two

conflicting pages cannot be turned on simultaneously.

How do you access the registers? To read or write to a register requires use of the "latch"; port 88H sets which register is to be accessed. Use the following routines to read/write a register:

BASIC	Z80	FUNCTION
OUT&H88,RR	LD A,RR OUT(88H),A	Sets register to be accessed, RR equals register number. Latch remains set to register until reset to another value.
VV=INP(&H90)	IN A,(90H)	Read from register. VV = variable used to store value.
OUT&H8C,WW	LD A,WW OUT(8CH),A	Write to register. WW = value to go to register.

With the exception of writing to Port B, you can read and write to any of the registers. Using the above routines you could create sounds in machine code, read the joysticks and of course change memory banks. The main point to remember when swapping banks is not to put the routine to do this on the bank that is being disabled. If it is, then whatever is on the new bank will then be executed, or the computer might hang in a dead loop.

THE PPI

Since I have discussed the PSG, I should also discuss the PPI. This controls a similar set of devices as the PSG does. It is accessed by ports A (98H), B (99H), C (96H) and word control register (97H).

The word control register sets the "mode" of the PPI and is initialy set to 10010010B by the ROM. This register is best left alone until you possess a technical manual for the 8255 PPI.

Port A is used to read several functions. Bits 0 to 3 read connected paddles (which so far as I know don't yet exist) and graphics tablet. When the tablet is conected bits 0 and 1 are set to "1"; when disconnected they are set to "0". Attempting to read a paddle or pad output is a difficult and complicated affair and there are routines in ROM to do this. 3280H (PDL) will read a paddle. 32BDH (PAD) will read a graphic tablet. In either case the first instruction is a call to 1AA9H which loads the USR argument into register A. By already doing this you can make calls to 3283H and 32C0H instead. PDL takes an argument from 1 to 4, PAD takes an argument from 0 to 3. PDL will return a integer number between 0 - 255 at F925H.

When PAD takes an argument of 0 it attempts to read the tablet. If the tablet is not there it will fall into a dead loop which can only be broken by resetting the computer. Checking the lower two bits of Port A before calling this routine will solve this problem. Assuming the tablet is connected, X and Y values are read and placed in FE2CH and FE2DH respectfully and FE25H is loaded with FFH. When the argument is 1 or 2 these locations are read and returned in F925H, so they are not updated until another PAD(0) occurs and the surface is touched. Despite a reference to PAD(3), I couldn't find any routine that detected a switch on the tablet. This is no great loss however as the tablet I possess has no switch!

Bits 4 and 5 of Port A indicate if the fire buttons on joysticks 1 and 2 are pressed. A "0" indicates contact, "1" indicates no contact.

Bit 6 is set by having either PLAY, FASTFORWARD or REWIND keys pressed on the cassette player: a "1" indicates none of these keys is pressed, a "0" indicates any of them are. This bit is used by BASIC to produce a 'PRESS PLAY / & RECORD' message. Bit 7 is used to read the cassette itself and its state is dependent upon what is written on that cassette.

Port B is used to read the keyboard matrix and will produce a simple eight bit value based on the keys pressed. Port C is used as a latch port and to output to the cassette. Bits 0 to 3 are used to select which keyboard row to "strobe". To read a row of keys, use the following procedures:

BASIC	Z80	Function
OUT &H96,RR	LD A,RR OUT (96H),A	Sets Row. RR = Row read.
VV=INP(&H99)	IN A,(99H)	Reads Row. VV = Value returned

A fuller discussion of this process will be found in part 8 of this series.

Bit 4 of port C controls the cassette motor. Sending a "1" to this bit turns it off, sending a "0" turns it on. Bit 5 is used to write information to the cassette tape while saving a program / data file / memory block. Bit 6 enables the 2nd audio channel on the cassette tape. Setting it to "1" will enable it, setting it to "0" will disable it. Finally, bit 7 mixes the sound data from the PSG, though I have no further information about this as yet.

MSX SCREEN 1

If you have played with an MSX machine you have noticed that the screen modes are different. SCREEN 1 is actually an implementation of Graphics mode I on the VDP, which was discussed in part 7 of this series. This is a natural games mode because it utilises simple character definitions, you can redefine the colours of blocks of characters and you can put sprites on the screen at the same time!

Listing MSX COLOR demonstrates the redefining of colour blocks while in this mode. It will only work on an MSX computer. This has been tested on both a Toshiba and modified SV computer (the SV ran an MSX converter program, which changes the basic but not the ports used by the computer) and works on both.

Those with MSX computers will also find two additional graphics statements: BASE and VDP. BASE is used to determine where the tables used by the video chip are. This is a function only but is very useful for implementing your own arrangement. VDP is actually a copy of what was last sent to the eight write-only video registers (0-7) and a copy of the VDP register which is read only (8). Changing these registers by means other than basic will probably invalidate these values. The advantage of this feature is that you can effectively read the current state of the video chip, making it easy to alter.

Earlier articles in this series (parts 3 - 7 to be precise) have dealt with the function and operation of the video chip and new readers who own MSX computers will find these in the S.A.U.G. 1983/84 year book; another good reason to buy it!

NEXT PART

In the next part I will talk about utilising the ROM routines

inherant in the Spectravideo ROM to your best advantage. This will be the final segment dealing with Machine code. To wet your appetite until then, listing OK will demonstrate one way to change BASIC. Written by Robert Brinkworth, it enables the user to place their own message instead of the normal "Ok" prompt.

OK

by : R. Brinkworth

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV 84).
Send $1 to S.A.U.G. for printout of article.

```
NA     10 REM "OK" by Robert Brinkworth
AM     20 REM  -  Dumps routine to high memory
AN     30 MS!=&HFFC0:ML%=15:GOSUB200
OJ     40 REM  -  C$ contains message: maximum           of 255 bytes long
          !
EA     50 C$="Prompt Message"
EP     60 REM -   Dumps C$ to memory
AP     70 FORL=1TOLEN(C$):POKE&HFFCF+L,ASC(MID$(C$,L,1)):NEXT
DE     80 REM -   Dumps linefeed & carraige              return + end of s
          tring
AN     90 MS!=&HFFCF+L:ML%=3:GOSUB200
FI    100 REM -  Redirects vector to above              routine
CO    110 MS!=&HFE8E:ML%=3:GOSUB200
DB    120 NEW
BM    200 FORI=0TOML%-1:READV$:POKEMS!+I,VAL("&h"+V$):NEXT:RETURN
CE    300 DATA CD,63,64,21,D0,FF,CD,05
FO    310 DATA 1B,E1,21,C4,09,E5,C9
BO    320 DATA 0D,0A,00
BN    330 DATA C3,C0,FF
END
```

COLOR.MSX (M.S.X. COMPUTERS ONLY.)

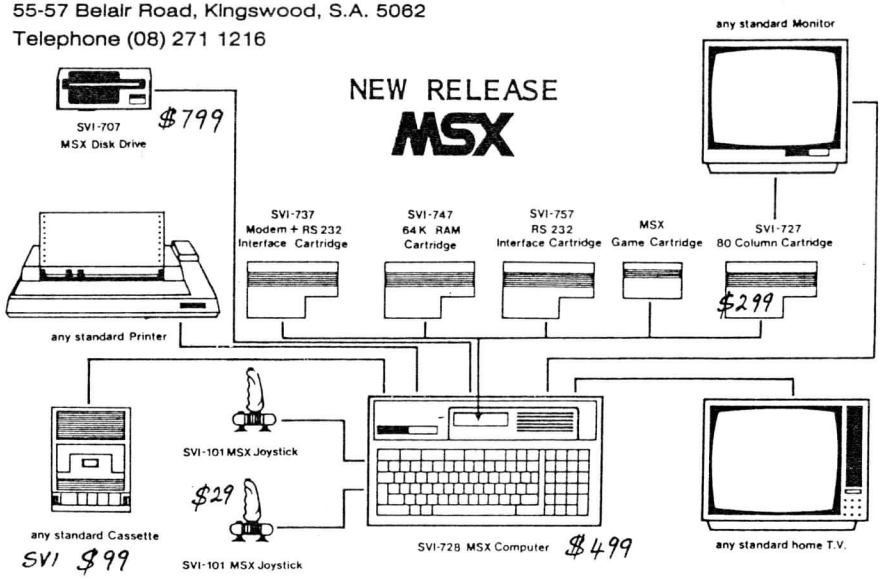
by : L.A. Dunning

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV 84).
Send $1 to S.A.U.G. for printout of article.

```
BB     10 REM MSX COLOR CHANGE                          by L.A.Dunning
BH     20 REM Works only on MSX computers
EG     30 SCREEN1:FORA=0TO255:VPOKEBASE(5)+A,A:NEXT
NH     40 LOCATE1,10:INPUT"Which set to change (0-31)";B$:B=ABS(VAL(B$)):I
          FB>31GOTO40
AK     50 LOCATE1,14:INPUT"Which colors (in hex)";V$:V=ABS(VAL("&h"+V$)):I
          FV>255GOTO50
CD     60 VPOKEBASE(6)+B,V:GOTO40
END
```

CLARKE ELECTRONICS

55-57 Belair Road, Kingswood, S.A. 5062
Telephone (08) 271 1216



THE SVI-728 SYSTEM PERIPHERAL MAP

TIP OF THE MONTH — CLARKE ELECTRONICS JUNE 85

If you have an 80 column card in your system you may have noticed the 50 cycle weave of the screen image. Coupled with the fact that SVI set up the screen with only one scanning line between each row of characters it promted me to fix the problem. Below is a BASIC program to modify the operating parameters of the 6845 CRT Controller used in the 80 column card. If a permanent fix is required the bios listing supplied by the users group is available.

```
150 FOR I = 0 TO 9                    'Set up loop to alter 10 values
160 OUT 80,I                          'Set up register No in 6845
170 READ A                            'Get data value from table
180 OUT 81,A                          'Set data value into 6845 register
190 NEXT I                            'Loop back and set another reg.
200 DATA 109,80,91,8,29,11,24,27,0,9  'Data table of register values
```

Starting from the first value in the DATA string to the last.....

REG.	Value		
0)	109	Horizontal Total Register	This is the one to fiddle with to fine tune the 50 cycle weave.
1)	80	Horizontal Displayed Reg.	Number of characters on the line
2)	91	Horizontal Sync Position	Positions the picture on the screen
3)	8	Horizontal Sync Width	Timing trimmer register
4)	29	Vertical Total Register	Together with REG. 5 determines
5)	11	Vertical Total Adjust	the vertical frequency
6)	24	Vertical Displayed Reg.	Number of character rows on screen
7)	27	Vertical Sync Position	Vertical position of picture
8)	0	Interlace Mode reg.	No interlace
9)	9	Max Scan line Addr. Reg.	Number of scan lines used for character

Best Regards Eric S. Clarke. Eric S. Clarke